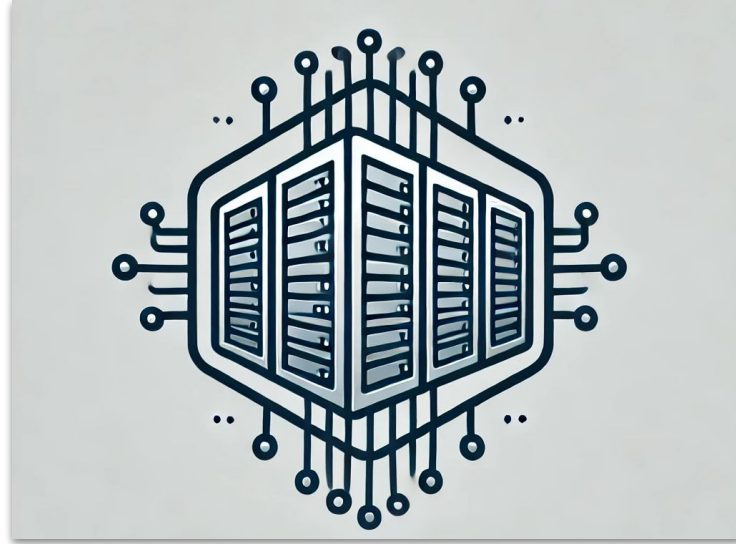


CSC 746: High Performance Computing

OMP Worksharing, Sync, Data Environment

1 Oct 2026



Today

Serial Reductions and Vectorization

Parallel Reductions

Example: Parallel Computation of Pi in OpenMP

OpenMP Parallel Loops, Worksharing

OpenMP Data Environment

Serial: Reductions and Vectorization

Code 1: Simple reduction

Function signature: `int sum(int N, int A[])`

Arguments:

- N is the number of items to process
- A[] is an array of ints

Code objective: compute the sum of all A[i] and return that sum

Compile the code with flags to enable automatic vectorization

Submit: cut-paste of your code, the report

On Perlmutter:
`module load cpu`

To compile the code manually:
`g++ -O2 -ftree-vectorize
-fopt-info-vec-all=report.txt -c
foo.cpp`

```

2 int sum_a(int N, int A[])
3 {
4     int sum=0;
5
6     for (int i=0;i<N;i++)
7         sum += A[i];
8
9     return sum;
10 }

```

Code to sum up A[] (left)

Vectorization report (below)

Did the loop vectorize?

```

39 [SFSU/openmp_examples] % g++-13 -O3 -fopt-info-vec-all -c ./vectorization.cpp
./vectorization.cpp:6:18: optimized: loop vectorized using 16 byte vectors
./vectorization.cpp:2:5: note: vectorized 1 loops in function.
./vectorization.cpp:9:11: note: ***** Analysis failed with vector mode V4SI
./vectorization.cpp:9:11: note: ***** Skipping vector mode V16QI, which would repeat the
analysis for V4SI
./vectorization.cpp:17:23: missed: couldn't vectorize loop

```



Code 2: Row reduction – vectorized

Function signature: `void sum(int N, int A[], int Y[])`

Arguments:

- N is the number of items to process
- A[] is an array of ints of length NxN (like CP3)
- Y[] is an array of ints of length N

Code objective: each Y[i] is the sum of all items in each row of A[i:.*]

This code has two nested loops

Compile the code with automatic vectorization enabled, examine the vectorization report

Submit: cut-paste of the code, cut-paste of the report.txt

On Perlmutter:
`module load cpu`

To compile the code manually:
`g++ -O2 -ftree-vectorize
-fopt-info-vec-all=report.txt -c
foo.cpp`

```

12 void sum_row(int N, int A[], int Y[])
13 {
14     // Assume A is NxN 1D array, Y[] is Nx1 1D array
15     // Objective here is to sum each row of A[i,*] and place result in Y[i]
16
17     for (int row=0; row<N; row++) {
18         Y[row] = 0;
19         for (int col=0; col<N; col++)
20             Y[row] += A[row*N + col];
21     }
22 }
23

```

Code to sum up rows of A[i,*],
place result into Y[i]

Vectorization report (below)

Did the loop vectorize?

How do we fix the code so it will
vectorize?

```

./vectorization.cpp:17:23: missed: couldn't vectorize loop
./vectorization.cpp:20:33: missed: not vectorized: complicated access pattern.
./vectorization.cpp:19:26: missed: couldn't vectorize loop
./vectorization.cpp:20:17: missed: not vectorized: complicated access pattern.
./vectorization.cpp:12:6: note: vectorized 0 loops in function.
./vectorization.cpp:22:1: note: ***** Analysis failed with vector mode V4SI
./vectorization.cpp:22:1: note: ***** Skipping vector mode V16QI, which would repeat t
he analysis for V4SI
./vectorization.cpp:20:23: missed: couldn't vectorize loop

```

```

24 void sum_row_v2(int N, int A[], int Y[])
25 {
26     // Assume A is NxN 1D array, Y[] is Nx1 1D array
27     // Objective here is to sum each row of A[i,*] and place result in Y[i]
28
29     for (int row=0; row<N; row++) {
30         int accum = 0;
31         for (int col=0; col<N; col++)
32             accum += A[row*N + col];
33         Y[row] = accum;
34     }
35 }

```

Code to sum up rows of A[i,*],
place result into Y[i]

Vectorization report (below)

Did the loop vectorize?

What happened?

```

./vectorization.cpp:29:23: missed: couldn't vectorize loop
./vectorization.cpp:32:32: missed: not vectorized: complicated access pattern.
./vectorization.cpp:31:26: optimized: loop vectorized using 16 byte vectors
./vectorization.cpp:24:6: note: vectorized 1 loops in function.
./vectorization.cpp:35:1: note: ***** Analysis failed with vector mode V4SI
./vectorization.cpp:35:1: note: ***** Skipping vector mode V16QI, which would repeat the analysis for V4SI

```

```
g++ -O2 -ftree-vectorize -fopt-info-vec-all -c svs.cpp
```

Vectorization Report Output Shows Use of SSE Instructions

```
10
11 int64_t
12 sum(int64_t N, uint64_t A[])
13 {
14     // printf(" inside sum_vector perform_sum, N=%lld \n", N);
15     int64_t i, accum=0;
16
17     for (i=0; i<N; i++)
18         accum += A[i];
19
20     return accum;
21 }
```

```
wes@perlmutter:login08: > g++ -O2 -ftree-vectorize -fopt-info-vec-all -c svs.cpp
svs.cpp:17:14: optimized: loop vectorized using 16 byte vectors
svs.cpp:12:1: note: vectorized 1 loops in function.
svs.cpp:20:11: note: ***** Analysis failed with vector mode V2DI
svs.cpp:20:11: note: ***** Skipping vector mode V16QI, which would repeat the analysis for V2DI
/opt/cray/pe/gcc/11.2.0/snos/include/g++/iostream:74:25: missed: statement clobbers memory: std::ios_base::Init::Init (&__ioinit);
/opt/cray/pe/gcc/11.2.0/snos/include/g++/iostream:74:25: missed: statement clobbers memory: __cxxabiv1::__cxa_atexit (__dt_comp , &__ioinit, &__dso_handle);
svs.cpp:21:1: note: ***** Analysis failed with vector mode VOID
```

```
g++ -O2 -march=native -ftree-vectorize -fopt-info-vec-all -c svb.cpp
```

Vectorization Report Output Shows Use of AVX/AVX2 Instructions

```
10
11 int64_t
12 sum(int64_t N, uint64_t A[])
13 {
14     // printf(" inside sum_vector perform_sum, N=%lld \n", N);
15     int64_t i, accum=0;
16
17     for (i=0;i<N;i++)
18         accum += A[i];
19
20     return accum;
21 }
```

```
wes@perlmutter:login08: ~ - ssh - g++ -O2 -march=native -ftree-vectorize -fopt-info-vec-all -c svb.cpp
```

```
svb.cpp
```

```
svb.cpp:17:14: optimized: loop vectorized using 32 byte vectors
```

```
svb.cpp:12:1: note: vectorized 1 loops in function.
```

```
svb.cpp:20:11: note: ***** Analysis failed with vector mode V4DI
```

```
svb.cpp:20:11: note: ***** Skipping vector mode V32QI, which would repeat the analysis for V4DI
```

```
/opt/cray/pe/gcc/11.2.0/snos/include/g++/iostream:74:25: missed: statement clobbers memory: std::ios_base::Init::Init (&__ioinit);
```

```
/opt/cray/pe/gcc/11.2.0/snos/include/g++/iostream:74:25: missed: statement clobbers memory: __cxxabiv1::__cxa_atexit (__dt_comp , &__ioinit, &__dso_handle);
```

```
svb.cpp:21:1: note: ***** Analysis failed with vector mode VOID
```

Parallel Reductions

Direct Sum Reduction – Concurrency and Independence

The sum operator is *associative*:

$$A + B + C + D = (A+B) + (C+D)$$

So we may safely compute subsets of the sum in any order then add them together.

E.g.

$$\sum_{i=0}^{n-1} A[i] = \sum_{i=0}^{n/2} A[i] + \sum_{i=n/2+1}^{n-1} A[i]$$

```
20 int64_t
21 sum(int64_t N, uint64_t A[])
22 {
23     int64_t i, accum=0;
24
25     for (i=0; i<N; i++)
26         accum += i;
27
28     return accum;
29 }
```

Direct Sum Reduction – Implementing parallelism

In OpenMP:

`#pragma omp parallel`

Defines a region of code that will run in parallel

`#pragma omp for`

Defines a worksharing construct, ie, a way to divide up the problem

Can combine them in one line as shown here on L25

```
20 int64_t
21 sum(int64_t N, uint64_t A[])
22 {
23     int64_t i, accum=0;
24
25     #pragma omp parallel for
26     for (i=0; i<N; i++)
27         accum += i;
28
29     return accum;
30 }
```

Direct Sum Reduction – Hazards?

In this code, there is a *data race* condition on L27

A data race occurs when more than 1 thread of execution is attempting to write to a given location (register, memory location)

The last one doing the write is the one that wins

```
20 int64_t
21 sum(int64_t N, uint64_t A[])
22 {
23     int64_t i, accum=0;
24
25     #pragma omp parallel for
26     for (i=0; i<N; i++)
27         accum += i;
28
29     return accum;
30 }
```

Option #1 for working around race conditions

Solution:

Use a “critical section” so that only one thread at a time may perform the write to the variable “accum”

Correctness: produces the correct answers

Pitfalls: defeats parallelism, this code is effectively serial

```
19
20 int64_t
21 sum(int64_t N, uint64_t A[])
22 {
23     int64_t i, accum=0;
24
25     #pragma omp parallel for
26     for (i=0;i<N;i++)
27     #pragma omp critical
28         accum += i;
29
30     return accum;
31 }
32
```



Option #2 for working around race conditions

Solution: each thread uses a different location in a shared array to store partial results

Correctness: produces the correct answer

Pitfalls: while there is no critical section, there is the potential for *false sharing* that will result in poor performance (poor scalability)

```
20 int64_t
21 sum(int64_t N, uint64_t A[])
22 {
23     int64_t i, accum=0;
24     #define MAX_THREADS 8
25     int64_t accum_buffer[MAX_THREADS] = {};
26
27     #pragma omp parallel
28     {
29         int my_thread = omp_get_thread_num();
30
31     #pragma omp for
32         for (i=0;i<N;i++)
33             accum_buffer[my_thread] += i;
34     }
35
36     for (i=0;i<MAX_THREADS;i++)
37         accum += accum_buffer[i];
38
39     return accum;
40 }
41
```



Option #3 for working around race conditions

Solution: use thread-local storage for local computations, then combine the partial results in parallel using a critical section

Correctness: produces the correct answer

Pitfalls: while there is a critical section, it is entered only once by each thread after all other computations are finished

This approach is the best solution for this particular problem: minimizes serialization, no false sharing

```
19
20 int64_t
21 sum(int64_t N, uint64_t A[])
22 {
23     int64_t i, accum=0;
24
25     #pragma omp parallel
26     {
27         int64_t tls=0;
28
29         #pragma omp for
30         for (i=0; i<N; i++)
31             tls += i;
32
33         #pragma omp critical
34             accum += tls;
35     }
36
37     return accum;
38 }
39
```



Option #3 for working around race conditions

Solution: use thread-local storage for local computations, then combine the partial results in parallel using a critical section

Correctness: produced

Pitfalls: while there entered only once other computations

This approach is the particular problem: no false sharing

These 3 options are all *manual* solutions.

OpenMP has built-in support for reduction operations so there are more options available to you.

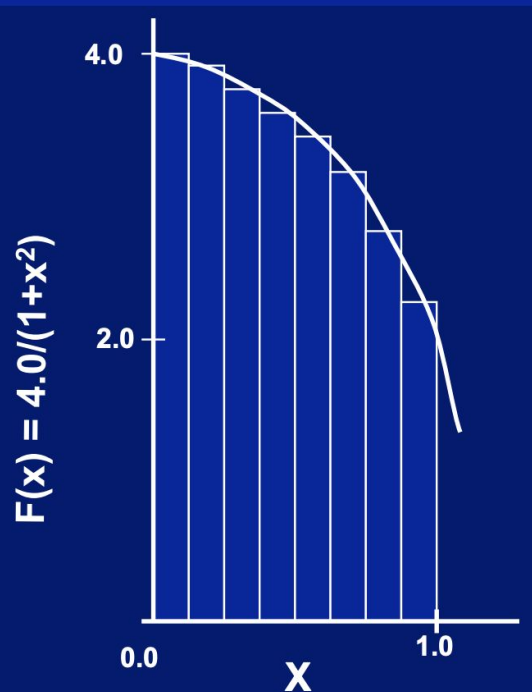
`#pragma omp parallel for reduction(+:foo)`

```
19
20 int64_t
21 sum(int64_t N, uint64_t A[])
22 {
23     int64_t i, accum=0;
24
25     #pragma omp parallel
26     {
27         int64_t tls=0;
28
29         #pragma omp for
30         for (i=0; i<N; i++)
31             tls += i;
32
33         #pragma omp critical
34         accum += tls;
35     }
36
37     return accum;
38 }
39
```



Example: Parallel Computation of π

Computing π



Mathematically, we know that:

$$\int_0^1 \frac{4.0}{(1+x^2)} dx = \pi$$

We can approximate the integral as a sum of rectangles:

$$\sum_{i=0}^N F(x_i) \Delta x \approx \pi$$

Where each rectangle has width Δx and height $F(x_i)$ at the middle of interval i .



Computing π

```
static long num_steps = 100000;
double step;
void main ()
{
    int i;    double x, pi, sum = 0.0;

    step = 1.0/(double) num_steps;
    x = 0.5 * step;
    for (i=0;i<= num_steps; i++){
        x+=step;
        sum += 4.0/(1.0+x*x);
    }
    pi = step * sum;
}
```



Identify Concurrency

```
static long num_steps = 100000;  
double step;  
void main ()  
{      int i;      double x, pi, sum = 0.0;  
  
        step = 1.0/(double) num_steps;  
        x = 0.5 * step;  
        for (i=0;i<= num_steps; i++){  
            x+=step;  
            sum += 4.0/(1.0+x*x);  
        }  
        pi = step * sum;  
}
```

Loop iterations
can in principle
be executed
concurrently



Identify Concurrency

```
static long num_steps = 100000;  
double step;  
void main ()  
{      int i;      double x, pi, sum = 0.0;
```

But is the loop as
written **safe for**
parallel execution?

Loop iterations
can in principle
be executed
concurrently

```
    step = 1.0/(double) num_steps;  
    x = 0.5 * step;
```

```
    for (i=0;i<= num_steps; i++){  
        x+=step;  
        sum += 4.0/(1.0+x*x);  
    }
```

Would this loop
vectorize??

```
    pi = step * sum;
```

```
}
```



Expose Concurrency (1)

Partition data
that must be
shared from
data **local** to
a task

This is called a **reduction**
... results from each
iteration accumulated
into a single global.

```
static long num_steps = 100000;
double step;
void main ()
{
    double pi, sum = 0.0;
    step = 1.0/(double) num_steps;

    int i; double x;
    for (i=0;i<= num_steps; i++){
        x = (i+0.5)*step;
        sum += 4.0/(1.0+x*x);
    }
    pi = step * sum;
}
```

Redefine x to
**remove loop
carried
dependence**



Expose Concurrency (2)

*Deal with the
reduction*

```
static long num_steps = 100000;
#define NUM 4 //expected max thread count
double step;
void main ()
{
    double pi, sum[NUM] = {0.0};
    step = 1.0/(double) num_steps;

    int i, ID=0; double x;
    for (i=0;i<= num_steps; i++){
        x = (i+0.5)*step;
        sum[ID] += 4.0/(1.0+x*x);
    }
    for(int i=0, pi=0.0;i<NUM;i++)
        pi += step * sum[i];
}
```

Common Trick: **privatize**
variable (sum) by
promoting a scalar to an
array indexed by the
number of threads

Need to add up
partial sums

Note: this is a serial program
... but I've now exposed the
concurrency so it can be
safely exploited in a parallel
execution.



Express Concurrency

SPMD

This style of parallelism is called "single program multiple data" (fixed number of threads execute same code, independently)

```
#include <omp.h>
static long num_steps = 100000;
#define NUM 4
double step;
void main ()
{
    double pi, sum[NUM] = {0.0};
    step = 1.0/(double) num_steps;
    #pragma omp parallel num_threads(NUM)
    {
        int i, ID; double x;
        ID = omp_get_thread_num();
        for (i=ID;i<= num_steps;i+=NUM){
            x = (i+0.5)*step;
            sum[ID] += 4.0/(1.0+x*x);
        }
        for(int i=0, pi=0.0;i<NUM;i++)
            pi += step * sum[i];
    }
}
```

Create NUM threads

Each thread executes code in the parallel block

Simple mod to loop to deal out iterations to threads

Note: creating NUM threads *inside* your program is not good practice. Specify concurrency outside your program in an environment variable.



Express Concurrency

```
#include <omp.h>
static long num_steps = 100000;
#define NUM 4
double step;
void main ()
{
    double pi, sum[NUM] = {0.0};
    step = 1.0/(double) num_steps;
    #pragma omp parallel num_threads(NUM)
    {
        int i, ID; double x;
        ID = omp_get_thread_num();
        for (i=ID;i<= num_steps; i+=NUM){
            x = (i+0.5)*step;
            sum[ID] += 4.0/(1.0+x*x);
        }

        for(int i=0, pi=0.0;i<NUM;i++)
            pi += step * sum[i];
    }
}
```

variables **declared**
inside a thread are
private to that
thread

automatic **variables**
declared outside a
parallel region are
shared between threads

Create NUM threads

Each thread executes code in the
parallel block

Simple mod to loop to deal out
iterations to threads

Note: creating NUM threads *inside*
your program is not good practice.
Specify concurrency outside your
program in an environment
variable.



Express Concurrency

```
#include <omp.h>
static long num_steps = 100000;
#define NUM 4
double step;
void main ()
{
    double pi, sum[NUM] = {0.0};
    step = 1.0/(double) num_steps;
    #pragma omp parallel num_threads(NUM)
    {
        int i, ID; double x;
        ID = omp_get_thread_num();
        for (i=ID;i<= num_steps; i+=NUM){
            x = (i+0.5)*step;
            sum[ID] += 4.0/(1.0+x*x);
        }
        for(int i=0, pi=0.0;i<NUM;i++)
            pi += step * sum[i];
    }
}
```

automatic **variables declared** outside a parallel region are **shared** between threads

Create NUM threads

Each thread executes code in the parallel block

Simple mod to loop to deal out iterations to threads

variables **declared inside** a thread are **private** to that thread

Where's
the bug?

Note: creating NUM threads *inside* your program is not good practice. Specify concurrency outside your program in an environment variable.



Express Concurrency

```
#include <omp.h>
static long num_steps = 100000;
#define NUM 4
double step;
void main ()
{
    double pi, sum[NUM] = {0.0};
    step = 1.0/(double) num_steps;
    #pragma omp parallel num_threads(NUM)
    {
        int nthreads = omp_get_num_threads();
        int i, ID;      double x;
        ID = omp_get_thread_num();
        for (i=ID;i<= num_steps; i+=nthreads){
            x = (i+0.5)*step;
            sum[ID] += 4.0/(1.0+x*x);
        }
    }

    for(int i=0, pi=0.0;i<NUM;i++)
        pi += step * sum[i];
}
```

NUM is a requested
number of threads,
but an OS can choose
to give you fewer.

Hence, you need to
add a bit of code to
get the actual
number of threads



π Program: performance results

- Original Serial pi program with 100000000 steps ran in 1.83 seconds.

OpenMP Pi Array + Serial sum



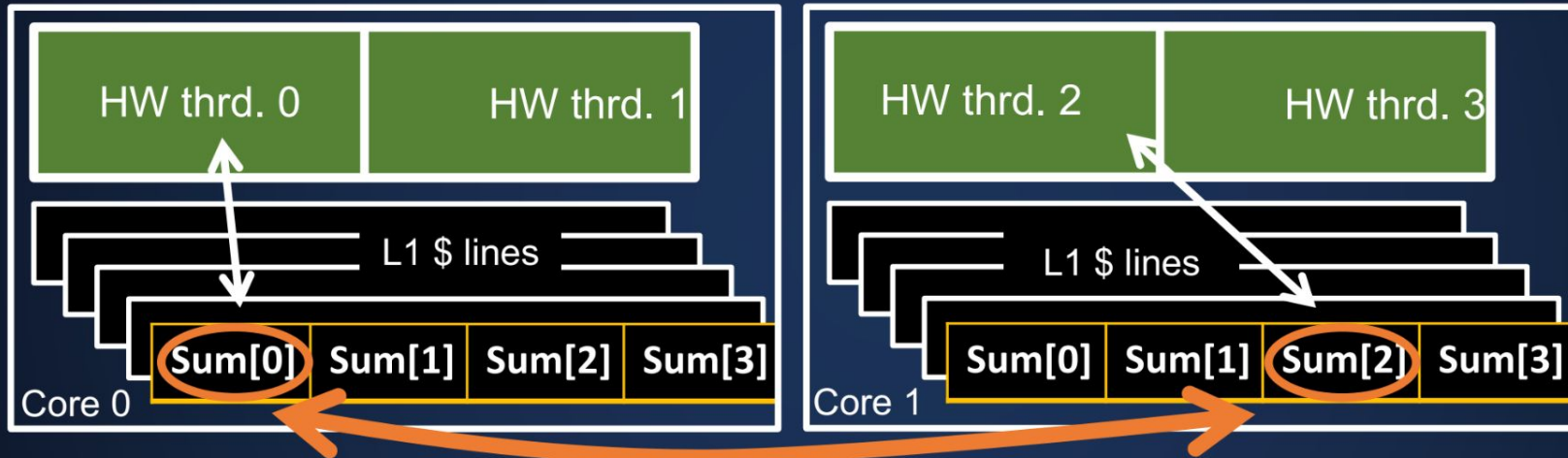
threads	Serial Sum	Speedup
1	1.86	1
2	1.03	1.8x
3	1.08	1.7x
4	0.97	1.9x

Why is the performance so poor?

*Intel compiler (icpc) with no optimization on Apple OS X 10.7.3 with a dual core (four HW thread) Intel® Core™ i5 processor at 1.7 Ghz and 4 Gbyte DDR3 memory at 1.333 Ghz.

π Program Performance Bottleneck

False sharing: Independent variables sit on the same cache line, each update will cause the cache lines to “slosh back and forth” between threads

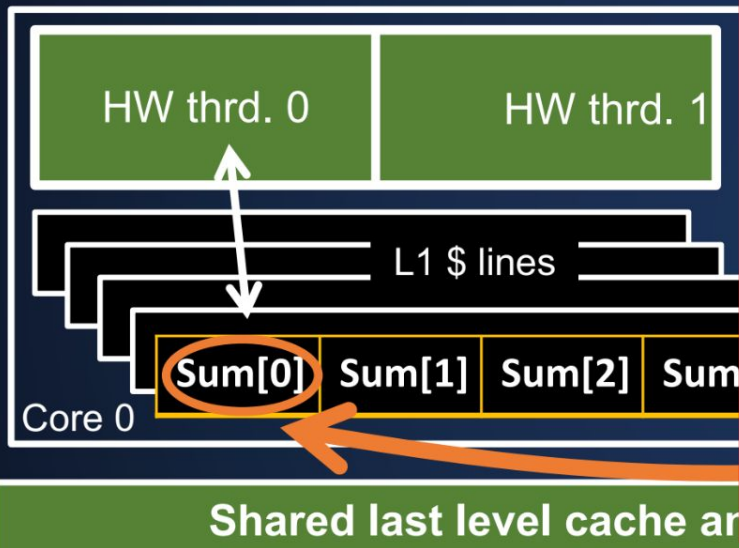


Shared last level cache and connection to I/O and DRAM

- Privatized scalars into an array to support parallelism... Writes to contiguous elements result in poor scalability.
- Solution: Pad arrays so elements you use are on distinct cache lines.

π Program Performance Bottleneck

False sharing: Independent variables
cause the cache lines to “slosh b



Q: How large is the an L1 cache line on the AMD Milan?

A: 64 bytes

```
#define CACHE_LINE_SIZE 64  
double sum[Nthreads * CACHE_LINE_SIZE]
```

```
# pragma omp parallel for  
for (i:0,N  
    sum[i*CACHE_LINE_SIZE] = compute(i);
```

- Privatized scalars into an array to support parallelism... Writes to contiguous elements result in poor scalability.
- Solution: Pad arrays so elements you use are on distinct cache lines.

Synchronization: “critical” regions

Only one thread at a time may enter a *critical region*

Threads wait
their turn –
only one at a
time calls
consume()



```
float res;  
#pragma omp parallel  
{  float B;  int i, id, nthrds;  
    id = omp_get_thread_num();  
    nthrds = omp_get_num_threads();  
    for(i=id;i<niters;i+=nthrds){  
        B = big_job(i);  
#pragma omp critical  
        res += consume (B);  
    }  
}
```

π program: Safely Update Shared Data

```
#include <omp.h>
static long num_steps = 100000;
#define NUM 4
double step;
int main ()
{
    double pi, sum=0.0;
    step = 1.0/(double) num_steps;
    #pragma omp parallel num_threads(NUM)
    {
        int i, ID; double x, psum= 0.0;
        int nthreads = omp_get_num_threads();
        ID = omp_get_thread_num();
        for (i=ID; i<= num_steps; i+=nthreads){
            x = (i+0.5)*step;
            psum += 4.0/(1.0+x*x);
        }
        #pragma omp critical
            sum += psum;
    }
    pi = step * sum;
}
```

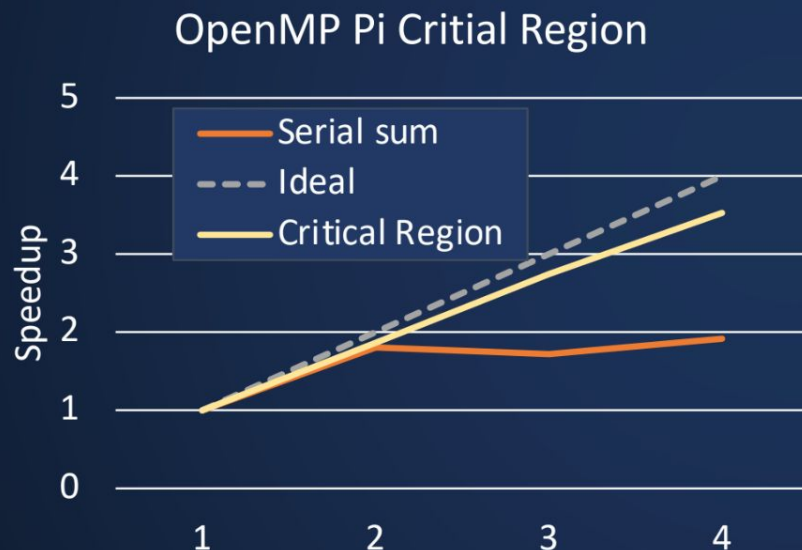
Replace array for sum
with a **local/private**
version of sum (**psum**) ...
no more false sharing

Use a **critical section** so only
one thread at a time can
update sum, i.e. you can
safely combine psum values



π Program Updated Performance Results

- Original Serial π program with 100000000 steps ran in 1.83 seconds.



threads	Serial Sum	Critical Region
1	1.86	1.87
2	1.03	1.00
3	1.08	0.68
4	0.97	0.53

*Intel compiler (icpc) with no optimization on Apple OS X 10.7.3 with a dual core (four HW thread) Intel® Core™ i5 processor at 1.7 Ghz and 4 Gbyte DDR3 memory at 1.333 Ghz.

π Program: Use omp to safely compute reduction

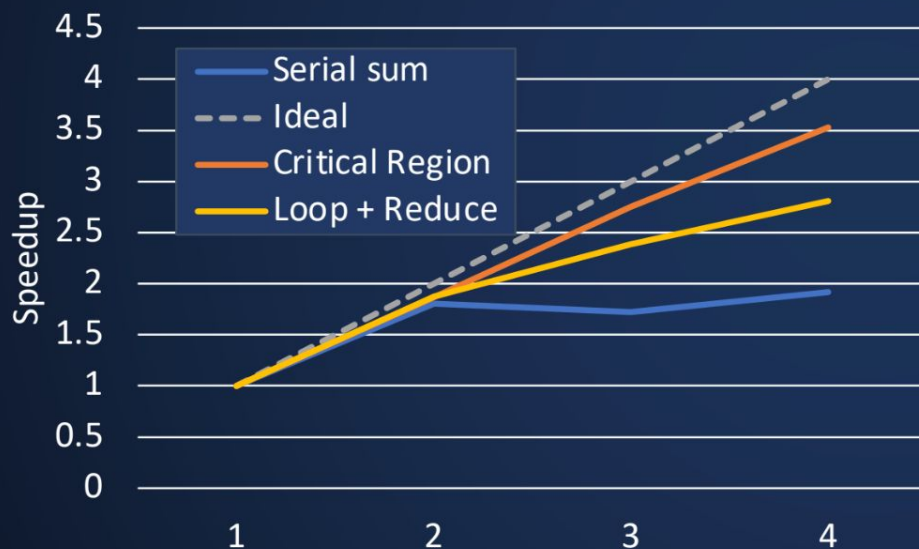
```
#include <omp.h>
static long num_steps = 100000;    double step;
void main ()
{  int i;    double x, pi, sum = 0.0;
   step = 1.0/(double) num_steps;
   #pragma omp parallel
   {
       double x;
       #pragma omp for reduction(+:sum)
       for (i=0;i< num_steps; i++){
           x = (i+0.5)*step;
           sum = sum + 4.0/(1.0+x*x);
       }
   }
   pi = step * sum;
}
```



π Program Updated Performance Results

- Original Serial π program with 100000000 steps ran in 1.83 seconds.

OpenMP π Critical Region



threads	Serial Sum	Critical Region	Loop + Reduction
1	1.86	1.87	1.91
2	1.03	1.00	1.02
3	1.08	0.68	0.80
4	0.97	0.53	0.68

Discussion

Serial sum:

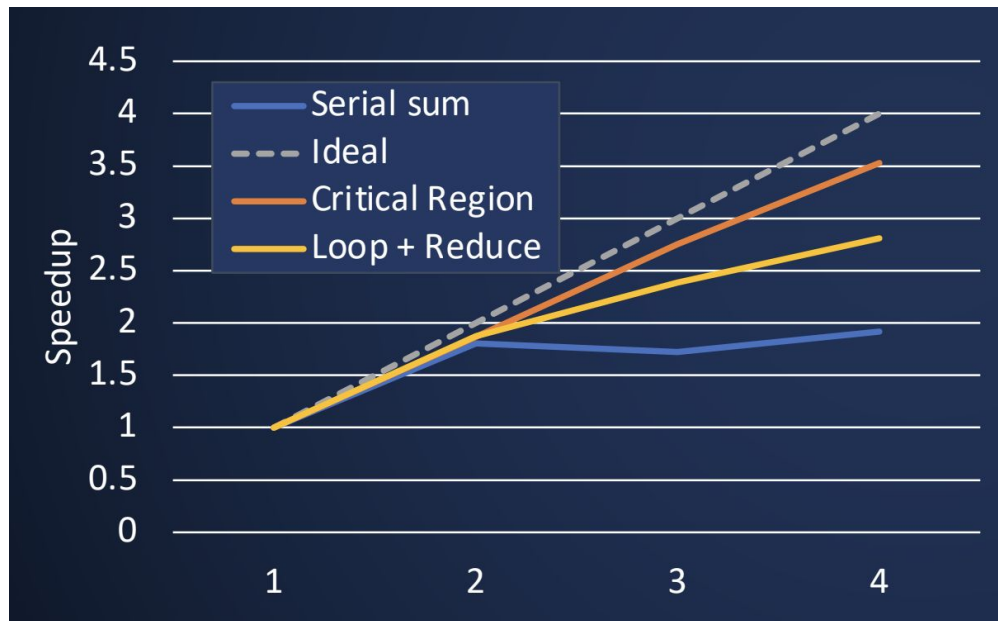
- False sharing: inner loop synchronization

Critical region:

- Independent computations, accumulate in local variables
- Serialize sum outside inner loop

Loop+reduce:

- Similar to critical region
- Relying on compiler optimizations: didn't seem to work out so well here



OpenMP Parallel Loops, Worksharing

The loop worksharing Constructs

- The loop worksharing construct splits up loop iterations among the threads in a team

```
#pragma omp parallel
{
  #pragma omp for
    for (l=0;l<N;l++){
      NEAT_STUFF(l);
    }
}
```

Loop construct
name:

- C/C++: for
- Fortran: do



The variable `l` is made “private” to each thread by default. You could do this explicitly with a “`private(l)`” clause

Loop worksharing Constructs

A motivating example

Source: Tim Mattson, Intel

Sequential code

```
for(i=0;i<N;i++) { a[i] = a[i] + b[i];}
```

OpenMP parallel region

```
#pragma omp parallel
{
    int id, i, Nthrds, istart, iend;
    id = omp_get_thread_num();
    Nthrds = omp_get_num_threads();
    istart = id * N / Nthrds;
    iend = (id+1) * N / Nthrds;
    if (id == Nthrds-1)iend = N;
    for(i=istart;i<iend;i++) { a[i] = a[i] + b[i];}
}
```

OpenMP parallel region and a worksharing for construct

```
#pragma omp parallel
#pragma omp for
    for(i=0;i<N;i++) { a[i] = a[i] + b[i];}
```



Combined parallel/worksharing construct

Source: Tim Mattson, Intel

- OpenMP shortcut: Put the “parallel” and the worksharing directive on the same line

```
double res[MAX]; int i;  
#pragma omp parallel  
{  
    #pragma omp for  
    for (i=0;i< MAX; i++) {  
        res[i] = huge();  
    }  
}
```

```
double res[MAX]; int i;  
#pragma omp parallel for  
    for (i=0;i< MAX; i++) {  
        res[i] = huge();  
    }
```

These are equivalent



Working with loops

- Basic approach
 - ◆ Find compute intensive loops
 - ◆ Make the loop iterations independent .. So they can safely execute in any order without loop-carried dependencies
 - ◆ Place the appropriate OpenMP directive and test

```
int i, j, A[MAX];  
j = 5;  
for (i=0; i< MAX; i++) {  
    j += 2;  
    A[i] = big(j);  
}
```

Note: loop index
“i” is private by
default

Remove loop
carried
dependence

```
int i, A[MAX];  
#pragma omp parallel for  
for (i=0; i< MAX; i++) {  
    int j = 5 + 2*(i+1);  
    A[i] = big(j);  
}
```



Nested loops

Source: Tim Mattson, Intel

- For perfectly nested rectangular loops we can parallelize multiple loops in the nest with the collapse clause:

```
#pragma omp parallel for collapse(2)
for (int i=0; i<N; i++) {
    for (int j=0; j<M; j++) {
        . . . . .
    }
}
```

Number of
loops to be
parallelized,
counting from
the outside

- Will form a single loop of length $N \times M$ and then parallelize that.
- Useful if N is $O(\text{no. of threads})$ so parallelizing the outer loop makes balancing the load difficult.



Reduction

- How do we handle this case?

```
double ave=0.0, A[MAX];  int i;
for (i=0; i< MAX; i++) {
    ave += A[i];
}
ave = ave/MAX;
```

- We are combining values into a single accumulation variable (ave) ... there is a true dependence between loop iterations that can't be trivially removed
- This is a very common situation ... it is called a “reduction”.
- Support for reduction operations is included in most parallel programming environments.



Reduction

- OpenMP reduction clause:
reduction (op : list)
- Inside a parallel or a work-sharing construct:
 - A local copy of each list variable is made and initialized depending on the “op” (e.g. 0 for “+”).
 - Updates occur on the local copy.
 - Local copies are reduced into a single value and combined with the original global value.
- The variables in “list” must be shared in the enclosing parallel region.

```
double ave=0.0, A[MAX];  int i;  
#pragma omp parallel for reduction (+:ave)  
for (i=0; i< MAX; i++) {  
    ave += A[i];  
}  
ave = ave/MAX;
```

Source: Tim Mattson, Intel



OpenMP: Reduction operands/initial-values

- Many different associative operands can be used with reduction:
- Initial values are the ones that make sense mathematically.

Operator	Initial value
+	0
*	1
-	0
min	Largest pos. number
max	Most neg. number

C/C++ only	
Operator	Initial value
&	~0
	0
^	0
&&	1
	0

Fortran Only	
Operator	Initial value
.AND.	.true.
.OR.	.false.
.NEQV.	.false.
.IEOR.	0
.IOR.	0
.IAND.	All bits on
.EQV.	.true.



Vector-matrix multiplication: $y = y + A * x$

Assume:

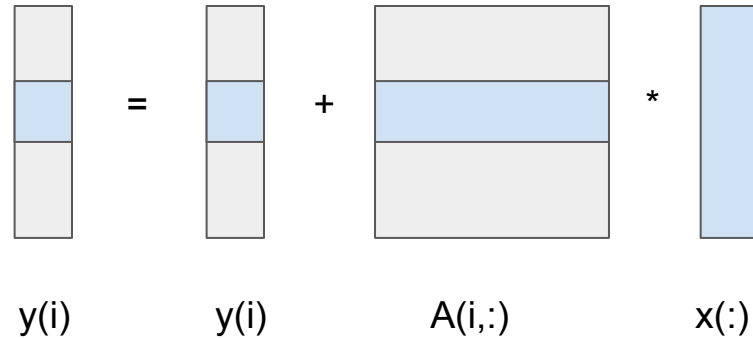
x , y are vectors of length n

A is an $n \times n$ matrix

for $i=1:n$ // 1-based indexing

for $j=1:n$

$$y[i] = y[i] + A[i,j] * x[j]$$



OpenMP Data Environment

Data environment: Default storage attributes

Source: Tim Mattson, Intel

- Shared Memory programming model:
 - Most variables are shared by default
- Global variables are SHARED among threads
 - Fortran: COMMON blocks, SAVE variables, MODULE variables
 - C: File scope variables, static
 - Both: dynamically allocated memory (ALLOCATE, malloc, new)
- But not everything is shared...
 - Stack variables in subprograms(Fortran) or functions(C) called from parallel regions are PRIVATE
 - Automatic variables within a statement block are PRIVATE.



Data sharing: Examples

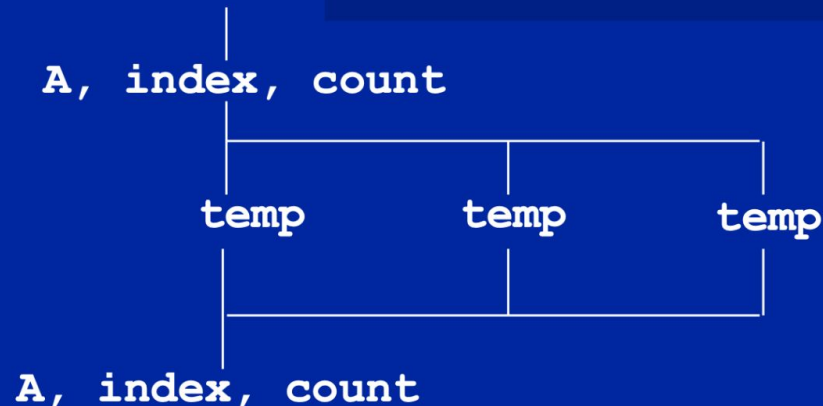
Source: Tim Mattson, Intel

```
double A[10];
int main() {
    int index[10];
    #pragma omp parallel
        work(index);
    printf("%d\n", index[0]);
}
```

A, index and count are shared by all threads.

temp is local to each thread

```
extern double A[10];
void work(int *index) {
    double temp[10];
    static int count;
    ...
}
```



Data Sharing: Private Clause

- **private(var)** creates a new local copy of var for each thread.
 - The value of the private copies is uninitialized
 - The value of the original variable is unchanged after the region

```
void wrong() {  
    int tmp = 0;  
    #pragma omp parallel for private(tmp)  
    for (int j = 0; j < 1000; ++j)  
        tmp += j;  
    printf("%d\n", tmp);  
}
```

tmp was not
initialized

tmp is 0 here

Source: Tim Mattson, Intel

Firstprivate Clause

- Variables initialized from shared variable
- C++ objects are copy-constructed

```
incr = 0;  
#pragma omp parallel for firstprivate(incr)  
for (i = 0; i <= MAX; i++) {  
    if ((i%2)==0) incr++;  
    A[i] = incr;  
}
```

Each thread gets its own copy
of incr with an initial value of 0

Source: Tim Mattson, Intel



Lastprivate Clause

Source: Tim Mattson, Intel

- Variables update shared variable using value from last iteration
- C++ objects are updated as if by assignment

```
void sq2(int n, double *lastterm)
{
    double x; int i;
    #pragma omp parallel for lastprivate(x)
    for (i = 0; i < n; i++){
        x = a[i]*a[i] + b[i]*b[i];
        b[i] = sqrt(x);
    }
    *lastterm = x;
}
```

“x” has the value it held for the “last sequential” iteration (i.e., for $i=(n-1)$)



Data Sharing:

A data environment test

Source: Tim Mattson, Intel

- Consider this example of PRIVATE and FIRSTPRIVATE

```
variables: A = 1, B = 1, C = 1  
#pragma omp parallel private(B) firstprivate(C)
```

- Are A,B,C local to each thread or shared inside the parallel region?
- What are their initial values inside and values after the parallel region?

Inside this parallel region ...

- “A” is shared by all threads; equals 1
- “B” and “C” are local to each thread.
 - B’s initial value is undefined
 - C’s initial value equals 1

Following the parallel region ...

- B and C revert to their original values of 1
- A is either 1 or the value it was set to inside the parallel region



Data Sharing: Default Clause

Source: Tim Mattson, Intel

- Note that the default storage attribute is **DEFAULT(SHARED)** (so no need to use it)
 - ◆ Exception: **#pragma omp task**
- To change default: **DEFAULT(PRIVATE)**
 - ◆ *each* variable in the construct is made private as if specified in a private clause
 - ◆ mostly saves typing
- **DEFAULT(NONE)**: *no* default for variables in static extent. Must list storage attribute for each variable in static extent. Good programming practice!

Only the Fortran API supports default(private).

C/C++ only has default(shared) or default(none).



The Mandelbrot Area program

Source: Tim Mattson, Intel

```
#include <omp.h>
# define NPOINTS 1000
# define MXITR 1000
void testpoint(void);
struct d_complex{
    double r;    double i;
};
struct d_complex c;
int numoutside = 0;

int main(){
    int i, j;
    double area, error, eps = 1.0e-5;
#pragma omp parallel for default(shared) private(c,eps)
    for (i=0; i<NPOINTS; i++) {
        for (j=0; j<NPOINTS; j++) {
            c.r = -2.0+2.5*(double)(i)/(double)(NPOINTS)+eps;
            c.i = 1.125*(double)(j)/(double)(NPOINTS)+eps;
            testpoint();
        }
    }
    area=2.0*2.5*1.125*(double)(NPOINTS*NPOINTS-
numoutside)/(double)(NPOINTS*NPOINTS);
    error=area/(double)NPOINTS;
}
```

```
void testpoint(void){
    struct d_complex z;
    int iter;
    double temp;

    z=c;
    for (iter=0; iter<MXITR; iter++){
        temp = (z.r*z.r)-(z.i*z.i)+c.r;
        z.i = z.r*z.i*2+c.i;
        z.r = temp;
        if ((z.r*z.r+z.i*z.i)>4.0) {
            numoutside++;
            break;
        }
    }
}
```

When I run this program, I get a different incorrect answer each time I run it ... there is a race condition!!!!

Debugging parallel programs

Source: Tim Mattson, Intel

- Find tools that work with your environment and learn to use them. A good parallel debugger can make a huge difference.
- But parallel debuggers are not portable and you will assuredly need to debug “by hand” at some point.
- There are tricks to help you. The most important is to use the `default(none)` pragma

```
#pragma omp parallel for default(none) private(c, eps)
for (i=0; i<NPOINTS; i++) {
    for (j=0; j<NPOINTS; j++) {
        c.r = -2.0+2.5*(double)(i)/(double)(NPOINTS)+eps;
        c.i = 1.125*(double)(j)/(double)(NPOINTS)+eps;
        testpoint();
    }
}
```

Using
`default(none)`
generates a
compiler
error that `j` is
unspecified.



Serial PI Program

Now that you understand how to modify the data environment, let's take one last look at our pi program.

```
static long num_steps = 100000;
double step;
int main ()
{
    int i;  double x, pi, sum = 0.0;

    step = 1.0/(double) num_steps;

    for (i=0;i< num_steps; i++){
        x = (i+0.5)*step;
        sum = sum + 4.0/(1.0+x*x);
    }
    pi = step * sum;
}
```

What is the minimum change I can make to this code to parallelize it?



Example: Pi program ... minimal changes

Source: Tim Mattson, Intel

#include <omp.h>

static long num_steps = 100000; double step;

void main ()

{ int i; double x, pi, sum = 0.0;
 step = 1.0/(double) num_steps;

#pragma omp parallel for private(x) reduction(+:sum)

for (i=0; i< num_steps; i++){
 x = (i+0.5)*step;
 sum = sum + 4.0/(1.0+x*x);

i private by
default

 }
 pi = step * sum;

}

For good OpenMP
implementations,
reduction is more
scalable than critical.

Note: we created a
parallel program without
changing any executable
code and by adding 2
simple lines of text!

Data sharing:

Changing storage attributes

- One can selectively change storage attributes for constructs using the following clauses*
 - **SHARED**
 - **PRIVATE**
 - **FIRSTPRIVATE**
- The final value of a private inside a parallel loop can be transmitted to the shared variable outside the loop with:
 - **LASTPRIVATE**
- The default attributes can be overridden with:
 - **DEFAULT (PRIVATE | SHARED | NONE)**
DEFAULT(PRIVATE) *is Fortran only*

All the clauses on this page apply to the OpenMP construct NOT to the entire region.

***All data clauses apply to parallel constructs and worksharing constructs except “shared” which only applies to parallel constructs.**

Source: Tim Mattson, Intel



Major OpenMP constructs we've covered so far

- To create a team of threads
 - ◆ `#pragma omp parallel`
- To share work between threads:
 - ◆ `#pragma omp for`
 - ◆ `#pragma omp single`
- To prevent conflicts (prevent races)
 - ◆ `#pragma omp critical`
 - ◆ `#pragma omp atomic`
 - ◆ `#pragma omp barrier`
 - ◆ `#pragma omp master`
- Data environment clauses
 - ◆ `private (variable_list)`
 - ◆ `firstprivate (variable_list)`
 - ◆ `lastprivate (variable_list)`
 - ◆ `reduction(+:variable_list)`

Where `variable_list` is a comma separated list of variables

Print the value of the macro
`_OPENMP`

And its value will be

`yyyyymm`

For the year and month of the spec the implementation used



The End



A Warner Bros.
PICTURE

